

Lit — Cheat Sheet

Guía rápida · Vite · Decoradores · Templates · Eventos · Directivas · Lifecycle

1 Comandos Básicos (Vite)

Iniciar un proyecto moderno de Lit con TypeScript usando Vite.

```
# Crear proyecto nuevo
npm create vite@latest my-lit-app -- --template lit-ts

# Instalar dependencias y ejecutar
cd my-lit-app
npm install
npm run dev
```

2 Decoradores Principales (Imports de `lit/decorators.js`)

Decorador	Angular Equivalente	Descripción
@customElement('my-tag')	@Component({selector: 'my-tag'})	Registra el Web Component en el navegador.
@property({ type: String })	@Input()	Propiedad pública (API del componente). Lit observa sus cambios.
@state()	Variables de clase	Estado interno privado. Al cambiar, vuelve a renderizar (render).
@query('#my-id')	@ViewChild()	Selecciona un elemento del Shadow DOM.

3 Template Syntax (El HTML)

Enlazar datos de la clase al template HTML dentro de la función `render()`.

- Data Binding (Texto): Evalúa expresiones dentro del HTML.

```
```html
<p>Hola, ${this.nombre}</p>
```
```

- Property Binding (.): Pasa datos complejos (objetos, arrays) a propiedades.

```
```html
<mi-componente .usuario=${this.userObj}></mi-componente>
```
```

- Attribute Binding (sin prefijo): Para atributos HTML nativos de tipo string.

```
```html
<div id=${this.dynamicId}></div>
```
```

- Boolean Attribute (?): Añade o quita el atributo entero si es true/false.

```
```html
<button ?disabled=${this.estaCargando}>Click</button>
```
```

- Event Binding (@): Escucha eventos del DOM o CustomEvents.

```
```html
<button @click=${this.handleClick}>Click</button>
```
```

4 Flujo de Control

Lit utiliza JavaScript puro, sin sintaxis de plantillas compleja.

- Condicionales (If / Else): Operadores ternarios.

```
```javascript
${this.isAdmin ? html<button>Borrar</button> : html<p>No autorizado</p>}
```
```

- Bucles (Loops): Usando `.map()` estándar de JavaScript.

```
```javascript

 ${this.items.map(item => html${item.name})}

```

...

## 5 Comunicación entre Componentes

La regla de oro de los componentes: Props Down, Events Up.

### ↓ Padre a Hijo (Propiedades)

El padre pasa el dato usando `.`, el hijo lo recibe con `@property()`.

```
// PADRE (Inyecta el objeto)
html`<flight-card .flight=${this.miVuelo}></flight-card>`

// HIJO (flight-card.ts)
@property({ type: Object }) flight?: Flight;
```

### ↑ Hijo a Padre (Eventos / Outputs)

El hijo usa el estándar CustomEvent. El padre lo escucha con `@`.

```
// HIJO (Dispara el evento)
seleccionar() {
 this.dispatchEvent(new CustomEvent('vuelo-seleccionado', {
 detail: { id: this.flight.id },
 bubbles: true, composed: true // Vital para atravesar el Shadow DOM
 }));
}

// PADRE (Escucha el evento)
html`<flight-card @vuelo-seleccionado=${this.manejarSeleccion}></flight-card>`

manejarSeleccion(event: CustomEvent) {
 console.log("Seleccionado:", event.detail.id);
}
```

## 6 Directivas Útiles (Imports de ``lit/directives/...``)

- `classMap`: Añade clases dinámicamente basado en booleanos (`ngClass`).

```
``typescript
import { classMap } from 'lit/directives/class-map.js';
const clases = { activo: this.isActive, error: this.hasError };
html`<div class=${classMap(clases)}></div>
...`
```

- `styleMap`: Añade estilos en línea dinámicamente (`ngStyle`).

```
``typescript
import { styleMap } from 'lit/directives/style-map.js';
const estilos = { color: 'red', marginTop: '10px' };
html`<div style=${styleMap(estilos)}></div>
...`
```

## 7 Ciclo de Vida (Lifecycle Hooks)

Los métodos que puedes sobrescribir en tu clase para engancharte a eventos del componente.

Hook	Angular Equivalente	¿Cuándo se ejecuta?
<code>connectedCallback()</code>	<code>ngOnInit</code>	El componente se añade al documento DOM. Ideal para peticiones HTTP (fetch) o añadir EventListeners globales.
<code>firstUpdated()</code>	<code>ngAfterViewInit</code>	El componente se ha renderizado por primera vez. Ideal para usar <code>@query</code> y manipular el DOM directamente.
<code>updated(changedProperties)</code>	<code>ngOnChanges</code>	Se ha terminado de renderizar una actualización causada por el cambio de una propiedad/estado.
<code>disconnectedCallback()</code>	<code>ngOnDestroy</code>	El componente se elimina del DOM. <b>Obligatorio</b> para limpiar <code>setIntervals</code> o Listeners globales.